Engineering Science and Technology, an International Journal 70 (2025) 102139

Contents lists available at ScienceDirect

Engineering Science and Technology, an International Journal

journal homepage: www.elsevier.com/locate/jestch

Full length article

Implementation of FPGA based 32-bit RISC-V processor

Kamer Kırali *, Can Bülent Fidan

Karabük University, Department of Mechatronic Engineering, Merkez, Karabük 78050, Turkey

ARTICLE INFO

Keywords:
FPGA
Verilog
RISC-V
ISA
Digital design
Computer architecture
Processors

ABSTRACT

This paper introduces the design and FPGA-based implementation of a compact and power-efficient 32-bit processor based on the RISC-V instruction set architecture, tailored for resource-constrained embedded systems. The processor features a simplified 4-stage pipeline that reduces hardware complexity and improves timing performance, making it suitable for low-area applications. A lightweight memory subsystem is integrated, including a 4 KB direct-mapped instruction cache and a 1 KB 2-way set-associative data cache, enhancing access efficiency without large memory demands. Peripheral support for UART, SPI, and PWM is provided via the Wishbone bus, enabling modular system integration. Arithmetic operations are implemented using hardware-friendly unsigned binary multiplication and non-restoring division algorithms to reduce logic utilization. Synthesized and evaluated on a Zybo Z7-20 FPGA, the processor achieves 1.35 CoreMark/MHz and 0.505 DMIPS/MHz, validating its suitability for compact embedded applications. The novelty of this study lies in the design of a compact RISC-V processor core that features a four-stage pipeline, occupies minimal area, and integrates a cache system (a 4 KB direct-mapped instruction cache and a 1 KB 2-way set-associative data cache), offering a balanced microarchitecture that combines low hardware cost with functional completeness. Unlike many previous works, the processor has been validated through real hardware tests with UART, SPI, and PWM modules, rather than relying solely on simulation. In addition, this work provides a detailed comparison against several open-source RISC-V cores such as CV32E40P and Ri5cy, demonstrating comparable or better performance with significantly lower area usage. Its compatibility with OpenRAM and OpenLane flows also makes it advantageous for ASIC implementations. These features collectively position this processor as a practical, low-cost alternative for embedded and edge-computing applications.

1. Introduction

Processors and microcontrollers have become fundamental components of today's world, spanning a broad range of applications from automation and military technologies to everyday computers, smartphones, and household appliances. As indispensable elements of the modern world, processors vary widely in terms of performance and power consumption. To achieve high performance with low power management, the RISC (Reduced Instruction Set Computer) architecture was developed. Among these architectures, RISC-V has gained prominence in both academic and commercial domains due to its open-source instruction set. RISC-V architecture remains true to the principles of RISC with its flexible design, minimalistic instruction set, and support for various extensions, which make it adaptable for diverse applications. The RISC-V Instruction Set Architecture (ISA) was developed by the Computer Science Department at the University of California, Berkeley, in 2010 [1]. Processors built on the RISC-V architecture have gained significant popularity in recent years. Most commercially successful ISAs have been proprietary, requiring licensing fees. Additionally, specific designs and implementations are often protected by patents and nondisclosure agreements. RISC-V stands out as the first modern, open-source ISA, promising an innovative development for the hardware industry. It has been further developed with contributions from major companies, including AMD, Google, Microsoft, and IBM. Since the initial versions of RISC-V were completed, numerous open-source tools and designs have been created and made publicly available. RISC-V attracts attention not only for being open-source but also for its potential as a powerful alternative to proprietary commercial ISAs like Intel, AMD, and ARM. While most academic designs are optimized for educational purposes, RISC-V has been developed for modern, real-world applications [2].

Embedded systems, integral to fields like medical devices, IoT, automotive, industrial automation, and artificial intelligence, demand processors that are both efficient and adaptable. The RISC-V instruction set architecture (ISA), with its modular and open-source nature, has

* Corresponding author.
E-mail addresses: kamer.kirali@gmail.com (K. Kırali), cbfidan@karabuk.edu.tr (C.B. Fidan).

https://doi.org/10.1016/j.jestch.2025.102139
Received 15 February 2025; Received in revised form 6 July 2025; Accepted 8 July 2025
Available online 7 August 2025
2215-0986/© 2025 The Authors. Published by Elsevier B.V. on behalf of Karabuk University. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

gained traction in embedded applications due to its flexibility and cost-effectiveness.

This study presents the design of a compact, 32-bit processor based on the RISC-V ISA, optimized for low area and power consumption, making it ideal for embedded systems implemented on Field Programmable Gate Arrays (FPGAs). The processor supports the RV32IM extension and integrates essential peripherals such as Universal Asynchronous Receiver–Transmitter (UART), Serial Peripheral Interface (SPI), and Pulse Width Modulation (PWM). Designed using Verilog at the Register Transfer Level (RTL), the processor is implemented on a Zybo Z7-20 FPGA board using the Vivado development environment.

To verify functionality, C++ programs were compiled in a Linux environment using the RISC-V GNU toolchain, converted to hexadecimal format, and tested on the processor. Debugging was facilitated through tools such as RISCOF and SPIKE at the RTL level, and simulation-based debugging was conducted using Venus and RARS. The processor's performance was evaluated using the Dhrystone benchmark, yielding a result of 0.5050 DMIPS/MHz.

In addition to the compact design, the processor's RAM structures were adapted for compatibility with the OpenRoad platform [3] and the OPENRAM framework [4], ensuring that the design is suitable for future integration with ASIC implementations.

The primary goal of this paper is to present an efficient, area-optimized RISC-V-based processor, demonstrating its applicability for resource-constrained environments such as embedded systems.

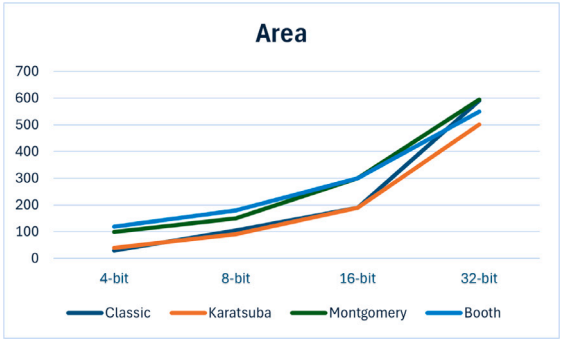
2. Related works

In their study, Yiyang Chang, Yiming Liu, Chong Peng, Jiarui Guo, and Yi Zhao focus on designing a RISC-V-based processor core using Verilog Hardware Description Language (HDL). In line with open-source principles, the processor design has been verified at both the sub-module and system levels, utilizing Verilator and GTKWAVE tools. During the initial tests, random instruction sequences were used to stress-test all components of the processor, identifying any errors. Low power consumption and high-performance configurations of the processor were implemented on a virtual machine using the System-C language. Various application tests, such as vector-vector addition, sorting, and encryption, were conducted to evaluate performance. Benchmark tests like Coremark and Dhrystone demonstrated that the designed core outperformed traditional microcontrollers like the ARM Cortex M3. Additionally, hardware-level prototyping was carried out with the Artix-7 FPGA, enabling the processor to operate at a speed of 200 MHz. The open-source toolchain and the absence of licensing costs make the core a cost-effective option, particularly for small-scale developers [5].

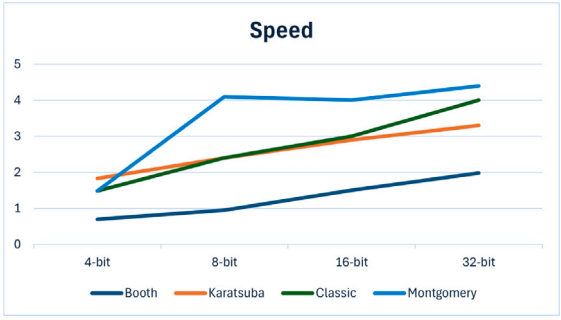
In his research, Wibowo stated that multiplication operations are costly and energy-consuming circuits. To compare more efficient multiplication algorithms for digital applications, four different algorithms were selected. Overall, this study compares the multiplication of two different numbers in an FPGA environment. According to the research results, the most efficient algorithm was found to be the Karatsuba algorithm, while the fastest algorithm was identified as the Booth algorithm. In Figs. 1(a) and 1(b) below there are speed and area comparisons [6].

Prem Kanjarbhat, Pooja Chandaragi, and Uday V Wali implemented a PWM signal application using a RISC-V processor. In their work, they aim to provide a comprehensive overview of the basic development tools in the RISC-V emulators and simulators ecosystem. The application tests were conducted on the Artix-7 FPGA board. They also indicated that a RISC-V CPU operating at a frequency of 100 MHz could be used for PWM functionality. This setup offers performance advantages such as low latency, high accuracy, and efficient resource utilization [7].

In the study conducted by Mohammad Awedh and Ahmed Mueen, a Microprogrammed Controller-based UART controller was implemented



(a)



(b)

Fig. 1. Illustrates the area occupied by multiplication algorithms and their respective speeds [6].

on an FPGA. This design is fully functional and synthesizable, using a top-down hierarchical design methodology based on Verilog. The de-sign was implemented on the Spartan-3E FPGA with Xilinx ISE Webpack 14.7. The application results show that the design operates at a maximum clock frequency of 218.248 MHz. The maximum clock frequency of the hardware-based UART controller was determined to be 192.773 MHz. When compared to the Read-Only Memory (ROM)-based method, the microprogrammed UART controller uses fewer bits, and therefore requires fewer storage elements. The functionality of the design was tested on the Digilent Basys2 Spartan3E FPGA board. The FPGA board was connected to a PC with a program that sends and receives large text and binary files (ranging from 50 MB to 100 MB). The program compared the sent bytes with the received bytes and verified that all transmitted bytes were correctly received. This study demonstrates that microprogrammed UART controllers can be successfully implemented on an FPGA. The design offers an efficient solution with a high clock frequency and low resource usage. The reduced FPGA resource usage compared to ROM-based methods highlights this design as an efficient and optimized option [8].

In their study, Negin Safari, Amirmahdi Joudi, Maryam Rajabalipanah, Zahra Ja-hanpeima, Hasan Sadeghzadeh, and Zainalabedin Navabi designed a RISC-V based processor core using Verilog HDL, adhering to open-source principles. This design was thoroughly tested through both modular verification and system-level tests. During the verification process, using Verilator and GTKWAVE tools, all components of the processor were subjected to stress testing, and various errors were identified. Performance analysis was conducted through various applications and comparative tests, with results showing that the processor outperformed traditional microcontrollers, such as the ARM Cortex M3. Furthermore, optimized configurations for low power consumption and high performance of the processor were simulated in a virtual environment using System-C. The hardware prototyping performed on the FPGA platform demonstrated that the processor operated at a speed of 200 MHz, and it was high-lighted that the lack

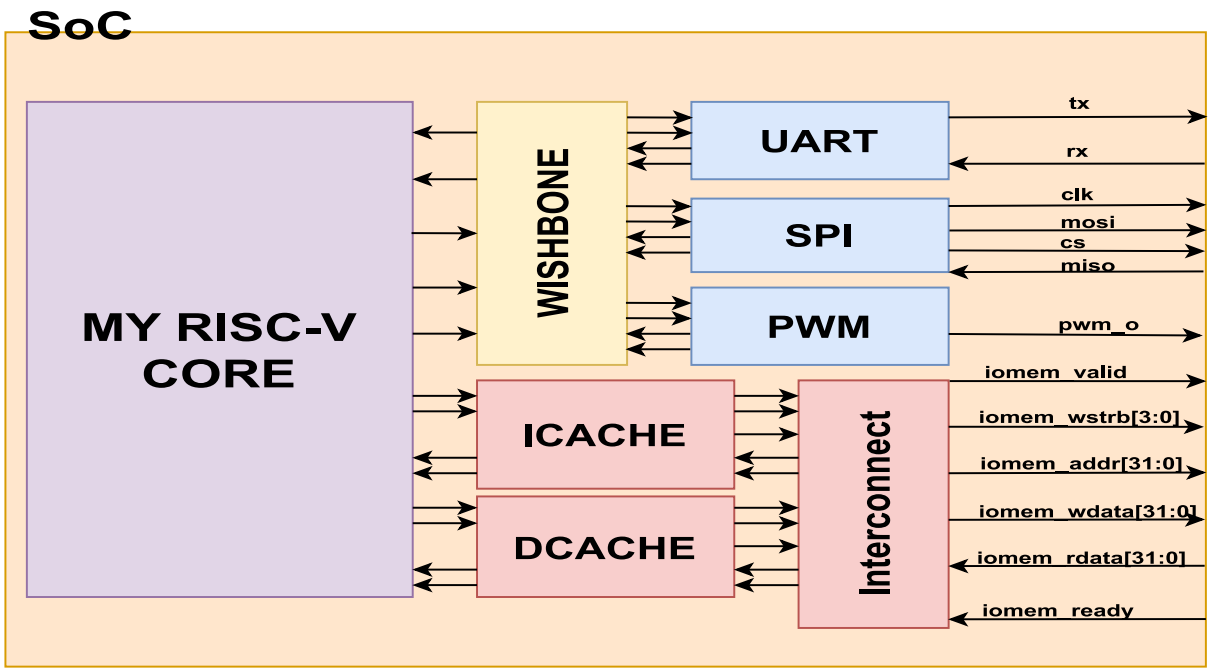


Fig. 2. General structure of the designed processor.

of licensing costs and the use of open-source tools provided significant cost advantages for small-scale developers [9].

This study evaluates various RISC-V cores on FPGA platforms using commonly employed benchmarks such as Dhrystone, CoreMark, and Embench. The results reveal significant variability in core performance, ranging from below 0.5 CoreMark/MHz to above 2.5 CoreMark/MHz. By conducting real hardware measurements, this study examines the application-level performance of the cores rather than relying on simulations. Among the cores, Taiga demonstrates the highest performance across all benchmarks, followed by Orca and VexRiscv. The benchmarks provide insights into the application-level performance of the cores, highlighting the impact of compiler op-timizations and memory configurations on the results [10].

In their study, Mario Gazziro, Jecel Mattos de Assumpção Junior, Oswaldo Hideo Ando Junior, Marco Roberto Cavallari, and João Paulo Carmo focused on the capability of FPGAs to provide a fully hardware-based solution with various input/output (I/O) devices. While the parallel operation of FPGA blocks facilitates meeting timing requirements, it also leads to the consumption of FPGA resources that could be allocated to other parts of the project. As an alternative, a small 'soft-core' processor capable of performing I/O operations in software is proposed. In this context, a new, compact soft-core processor optimized for FPGA resources for I/O purposes, named Baby8, has been developed. Baby8, with an instruction set aimed at 8-bit applications, achieves approximately 57 MHz maximum clock frequency and around 2 mW power consumption while using minimal FPGA resources. Additionally, compared to other soft-core processors, it provides competitive performance, reducing FPGA resource usage by half [11].

3. Materials and Methods

The Zybo Z7-20 FPGA board was used for testing the designed processor in the study. The Zybo Z7-20 board features an FPGA chip built on the Zynq 7000 family. It is a development board with various features that benefit embedded system software and digital design. Due to the abundance of resources it contains, it provides sufficient space for a wide range of designs. The specifications of the XC7Z020-1CLG400C chip in the Zybo Z7-20 are shown in The Zybo Z7-20 FPGA board was utilized for testing the RISC-V-based processor designed in this study. Featuring the XC7Z020-1CLG400C chip from the Zynq-7000 family, the Zybo Z7-20 offers 53,200 LUTs, 140 BRAM blocks, and a maximum clock frequency of 667 MHz, making it suitable for low-power embedded system designs. The board was configured using Vivado 2018.1, with the processor design occupying approximately 30% of available LUTs and 10% of BRAMs. These characteristics align with the processor's compact and area-optimized design objectives. Detailed specifications of the XC7Z020-1CLG400C chip are presented in Table 1.

Table 1
XC7Z020-1CLG400C features.

XC7Z020-1CLG400C	Quantity
Logic slices	13.300
6 input LUTS	53.200
Flip-Flops	106.400
Block Ram	630KB
DSP Slices	220
External Clock	125 MHz

To implement the proposed low-area and low-power RISC-V processor, the entire system was described using the Verilog hardware description language (HDL). Verilog enables precise control over hardware structures, making it suitable for area-efficient digital design targeting FPGA and ASIC platforms.

Xilinx Vivado was employed as the primary design and synthesis environment. Vivado offers advanced features such as high-level synthesis (HLS), place-and-route optimization, and bitstream generation. These capabilities were leveraged to reduce both logic utilization and dynamic power consumption during the design flow.

For software-level verification and architectural testing, the Venus simulator was used due to its support for the RV32IM instruction set. Its browser-based interface allows real-time tracking of register states, which significantly accelerated early-stage debugging and functional validation of the instruction execution pipeline.

RISCOF, an open-source RISC-V compliance framework, was utilized within an Ubuntu environment to ensure architectural conformance. Through comprehensive test suites, RISCOF helped detect and eliminate instruction-level discrepancies, which is crucial for ensuring

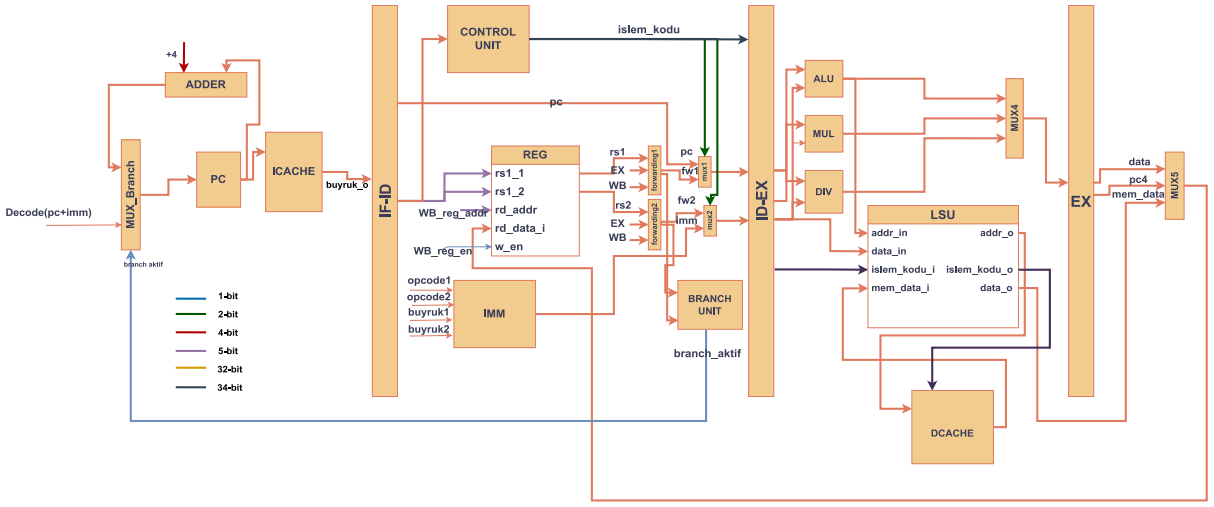


Fig. 3. Data path of designed core.

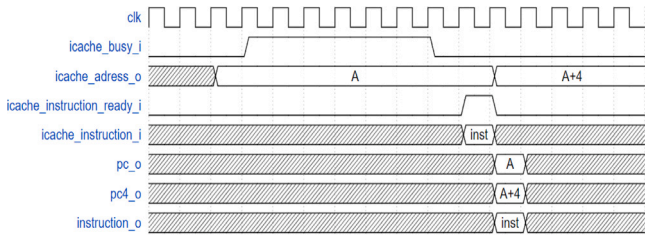


Fig. 4. FETCH phase signal representation.

correctness in low-power processor designs where redundant logic must be minimized [12].

The proposed processor, designed based on the RISC-V architecture, implements the RV32IM extensions and adopts a four-stage pipelined architecture. An overview of the system is provided in Fig. 2. Communication between the processor and peripheral components is achieved via the Wishbone bus protocol, ensuring reliable and high-speed data transfer. The design includes commonly used peripherals in embedded systems such as UART, SPI, and PWM, enabling comprehensive functional testing on physical platforms like FPGAs.

The four-stage pipeline structure enables the processor to execute one instruction per clock cycle, thereby increasing overall throughput. While deeper pipelines can support higher clock frequencies, they also introduce increased control complexity and potential branch penalties. In contrast, shallower pipelines simplify the control logic but require more operations per stage, potentially elongating the critical path and reducing performance. A four-stage pipeline strikes a practical balance between performance, hardware complexity, and energy efficiency.

To enhance memory access efficiency, the processor incorporates two cache types—an instruction cache and a data cache. These caches are designed to improve access latency and processor throughput by allowing direct interaction with main memory. A bridge module manages the data flow between main memory and both caches. Since the main memory employs a single input–output data line, this module consolidates communication paths from the caches to maintain proper data flow.

The processor core supports RV32IM extensions and follows a four-stage pipeline consisting of FETCH, DECODE, EXECUTE, and WRITE BACK stages. The "32" in RV32IM indicates 32-bit data path width. While the processor supports the full RV32I instruction set, it does not include the Control and Status Register (CSR) unit; thus, CSR-related instructions are not supported. With the M extension, the processor can perform arithmetic operations such as unsigned binary multiplication and non-restoring division. A schematic representation of the core structure is provided in Fig. 3.

3.1. Fetch

During the FETCH stage, the core retrieves instructions from memory. These instructions are then forwarded to the DECODE stage, where a register tracking the Program Counter (PC) and branch decisions is used. In this stage, the system interfaces with the instruction cache: the PC provides the address, and the instruction is fetched following an "instruction send" request. A signal-level representation of this stage is shown in Fig. 4.

3.2. Decode

The DECODE stage is where the incoming instruction is decoded. In this stage, there are components such as the Control Unit, Register File, Branch Unit, and Immediate Generator. The instruction received from the FETCH stage enters the DECODE stage, where it is analyzed by the Control Unit and converted into an operation codespecific signal. This code performs all core control operations, such as MUX control, determining whether data will be written to the Register File, and whether data will be written to memory. The meaning of the bits of the 31 bit process code designed and output from the control unit is shown in Fig. 5 below.

3.3. Execute

The EXECUTE stage is where the incoming data is processed or data is fetched from the memory unit. In this stage, units such as ALU, MUL, DIV, and LSU are involved. The ALU is responsible for performing basic arithmetic calculations in a single cycle for RV32I operations and is designed to meet the requirements of instructions in the RISC-V architecture. No special algorithm is used for addition, but custom algorithms are preferred for multiplication and division operations instead of operators like "*" and "/". The Unsigned Binary Multiplication algorithm is used for multiplication, and the Restoring Division algorithm is used for division.

The LSU (Load Store Unit) in the system handles communication with the data cache. Potential data is properly aligned in the LSU before being sent to or retrieved from the cache, and then forwarded to the next stage, the Write Back stage.

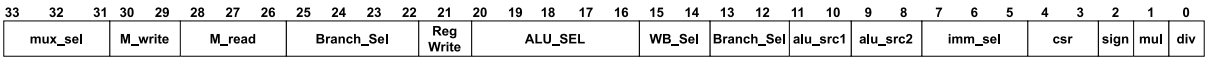


Fig. 5. Process code.

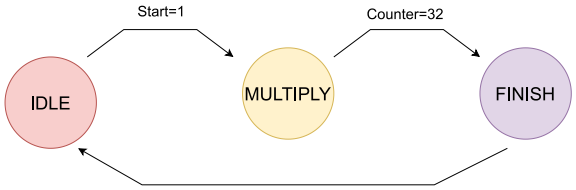


Fig. 6. Finite state machine diagram of multiplication algorithm.

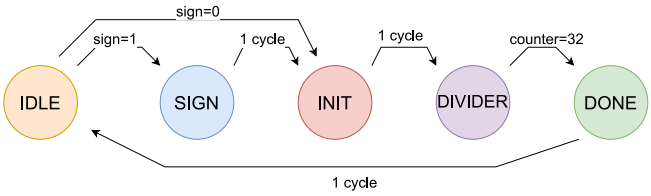


Fig. 7. Finite state machine diagram of division operation.

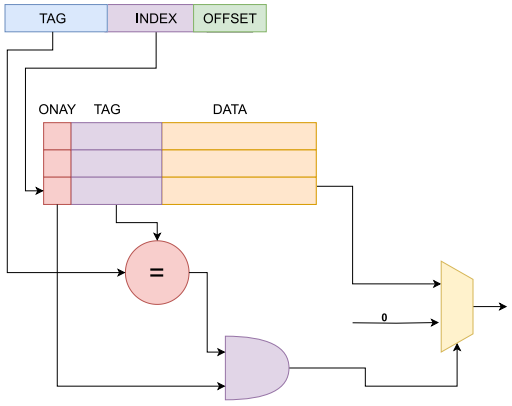


Fig. 8. Direct mapped cache structure.

3.3.1. Multiplication unit

To provide efficient arithmetic operations in resource-constrained embedded systems, the unsigned binary multiplication algorithm has been integrated into the processor's execution unit. This algorithm is preferred due to its low hardware complexity, simplicity, and deterministic timing behavior. In FPGA-based designs, considering limited logic resources and power consumption, this algorithm is highly suitable. Instead of complex and control-intensive methods, the unsigned multiplication technique offers fixed-latency and predictable performance while reducing the load on the control unit. This approach not only decreases the area footprint of the Arithmetic Logic Unit (ALU), but also contributes positively to FPGA synthesis and timing results. For the multiplication operation, the Unsigned Binary Multiplication algorithm is utilized. This algorithm is designed using a finite state machine (FSM) with three states. The multiplication operation is completed in 32 cycles, and when including FSM states, the operation ends in 34 cycles. Since the RISC-V ISA also supports signed multiplication operations, when signed numbers need to be multiplied, the 32nd bits of the numbers to be multiplied are stored in special registers. After the multiplication operation is complete, the registers holding the sign bits are subjected to an XOR operation to determine whether the result is positive or negative. If the XOR result is one, the result is negative; if zero, it is positive. Based on this, the result of the multiplication operation is inverted if necessary, according to the XOR outcome. Fig. 6 shows the finite state structure of the multiplication algorithm.

Similarly, for the multiplication operation, the Unsigned Binary Multiplication algorithm was preferred due to its minimal hardware complexity and predictable performance. More complex algorithms such as Booth multiplication or Wallace Trees offer higher performance but come with increased hardware complexity and power consumption, making them less suitable for embedded systems and FPGA-based designs. The Unsigned Binary Multiplication algorithm provides a simple and efficient solution in hardware, reducing the load on the control unit, simplifying timing closure, and offering deterministic behavior, which is ideal for low-power and area-constrained applications.

3.3.2. Division unit

The non-restoring division algorithm was selected in this processor design to handle arithmetic division operations due to its efficiency compared to other division algorithms, such as the classical restoring division method. While the restoring division method requires additional restoration steps during computation, the non-restoring algorithm eliminates these steps, simplifying the process. This reduction in control logic leads to decreased hardware complexity and improved execution time. In resource-constrained environments like embedded systems and FPGA-based designs, where area and timing constraints are critical, the non-restoring algorithm provides a more efficient solution [13].

Compared to other division algorithms such as SRT (Signed Digit) division or long division algorithms, the non-restoring division algorithm stands out due to its lower hardware requirements and faster execution time. SRT division offers faster results for specific number types but requires more complex control logic and larger hardware resources, making it less suitable for FPGA-based or embedded systems. Long division algorithms, on the other hand, can be inefficient in hardware and result in longer execution times and higher resource usage. The non-restoring algorithm strikes a balance between efficiency and simplicity, making it an ideal choice for low-area and low-power designs.

In this design, the non-restoring division algorithm is completed in a total of 35 cycles, and the method for storing the last bit sign used in the multiplication algorithm has also been incorporated to maintain consistency. The finite state structure of the division algorithm is shown in Fig. 7.

3.4. Instruction cache

Fig. 8 illustrates the structure of the instruction cache (ICACHE), which is designed with a capacity of 4 KB using the Direct Mapped Cache architecture. The selection of a 4 KB cache size is a deliberate balance between limited on-chip memory resources in FPGA platforms and the need to reduce instruction fetch latency. Smaller caches, such as 1 KB or 2 KB, would result in higher miss rates, increasing access to slower main memory, while significantly larger caches could exceed resource constraints or introduce timing complexity. In this context, 4 KB serves as a practical compromise that maintains system efficiency without overburdening the logic fabric or increasing power consumption disproportionately.

The decision to implement a Direct Mapped Cache structure is also guided by the processor's low-area, low-power design philosophy. Compared to more complex structures such as Set-Associative or Fully Associative Caches, the Direct Mapped approach minimizes hardware overhead and ensures deterministic access times-traits that are highly

Table 2
Cache address bit characteristics.

TAG	INDEX	OFFSET
31 30 29 28 .....13 12	11 10 9 8 7 6 5 4 3 2	1 0

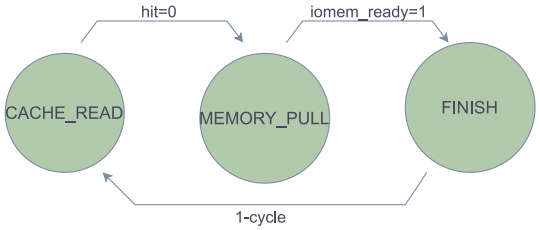


Fig. 9. ICACHE finite state machine structure.

valuable in real-time embedded systems. While Set-Associative caches may offer improved hit rates by reducing conflict misses, they introduce additional comparator logic, multiplexers, and tag matching complexity, which could negatively affect synthesis timing and power draw on FPGA-based platforms. Therefore, the Direct Mapped strategy provides a simpler and more predictable solution that aligns well with the modular and efficiency-driven architecture of the RISC-V processor used in this work [14].

The concepts of Tag and Index in the address work according to Table 2. The Offset section is not used. The reason for this is the absence of compressed instructions and the lack of unaligned memory units. Since there are no compressed instructions, none of the branches can target intermediate values of 32-bit lines. Therefore, there is no need for offset bits. Additionally, in the Fetch stage, the incoming address values will always be multiples of 4, so there is no need for them.

As shown in Fig. 9, this structure is designed with a finite state machine (FSM) logic and consists of three stages: CACHE_READ, MEMORY_PULL, and FINISH. In the CACHE_READ stage, a request for an instruction is sent from the core to the ICACHE. If a hit is achieved in the ICACHE, the instruction at the requested address is transmitted to the core; however, if a hit is not achieved, the FSM transitions to the MEMORY_PULL stage and sends a request to fetch the data at the requested address from the main memory. The main memory returns the data with a ready signal, and the FSM moves to the FINISH stage. After one cycle, the FSM returns to the CACHE_READ stage, and the data retrieved from the main memory is sent to the core with a hit 1 signal. The direct-mapped cache architecture was chosen for the instruction cache primarily because instruction accesses tend to be sequential and predictable. Since most programs execute instructions in order, the chance of cache conflicts is significantly lower, eliminating the need for a more complex associative structure. Moreover, the simplicity of the direct-mapped cache leads to lower hardware overhead and faster access times. These advantages are crucial for resource-constrained platforms such as embedded systems and FPGA implementations, where minimizing area and latency is a priority. This makes the direct-mapped architecture a practical and efficient choice for instruction caching.

3.5. Data cache

Fig. 10 shows the designed DCACHE structure. The data cache is implemented using a 2-way set-associative design and consists of two separate memory units, each with a capacity of 1 KB. Each way includes two independent RAM blocks with an 8-bit data width. These RAMs are divided into four subsections to enable byte- and half-word-level write operations across data rows. This structure improves memory access efficiency and provides flexibility in data handling. Fig. 11 illustrates the finite state machine (FSM) structure developed for the

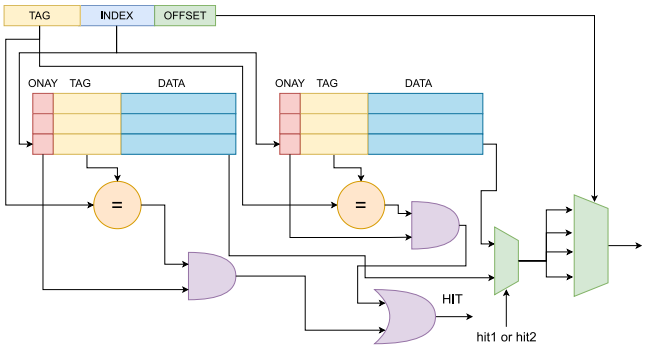


Fig. 10. Structure of DCACHE.

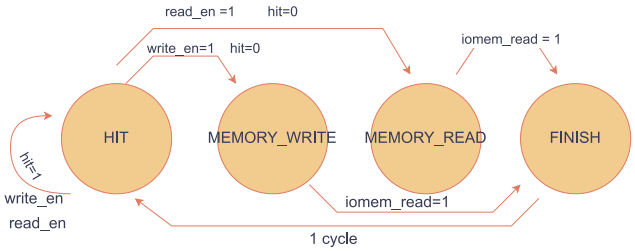


Fig. 11. DCACHE's Finite State Machine Structure.

data cache (DCACHE). According to the FSM, during a HIT, any read or write operation is executed directly in the cache and completes within a single clock cycle. If a HIT is not achieved, the processor is halted, and the system proceeds to the next phase. If the request is a read, the system transitions to the MEMORY_READ phase; for a write request, it moves to the MEMORY_WRITE phase. In these phases, data is transferred to or from the main memory accordingly, and the FSM then proceeds to the FINISH state. After one clock cycle, the system returns to the HIT phase and resumes normal operation.

When the system enters the MEMORY_READ phase, data is fetched from the main memory and written to the corresponding cache address. A dirty_bit is used to indicate data in the cache that has been updated but not yet written back to main memory. This ensures that any updated cache data is synchronized with the main memory upon new requests.

The system also features a 2-bit Least Recently Used (LRU) replacement policy. This algorithm determines which of the two cache lines in a set was used less recently and places the new data into the less frequently used line. This improves cache utilization and enhances data access performance.

Since the main memory bus has only a single data path for both input and output, a multiplexer-based interconnect system is used to manage communication between the instruction cache and the data cache. All multiplexers initially operate in INSTRUCTION mode. When a request from the data cache occurs, the system does not switch to DATA mode until the instruction transaction is complete. Afterward, the multiplexers automatically switch to DATA mode and begin transmitting data to or from the main memory. This structure ensures timely and accurate data transmission between the processor and memory.

The choice of a two-way set-associative structure for the data cache is primarily motivated by the need to reduce conflict misses, which are common in direct-mapped caches. In direct-mapped designs, blocks mapping to the same index continually replace each other, especially in loop-heavy access patterns, leading to degraded performance.

Two-way set-associative caching allows each set to store two blocks simultaneously, enabling different memory addresses with the same index but different tags to coexist. This greatly reduces conflict-based

Table 3
Memory map of UART.

Address	Name	Explanation
0x20000000	Uart_ctrl	Control register
0x20000004	Uart_status	Status register
0x20000008	Uart_rdata	Data read register
0x2000000c	Uart_wdata	Data write register

evictions and improves data access efficiency. Additionally, using a replacement algorithm such as LRU contributes to more effective cache management under irregular access conditions.

3.6. Wishbone bus

Communication between the processor core and peripheral devices is crucial for overall system efficiency. To achieve fast and efficient communication, the Wishbone bus protocol was adopted. Its modular structure facilitates easy interfacing between system components, simplifying design and increasing flexibility. Peripheral modules such as UART, SPI, and PWM were designed to comply with this protocol. The Wishbone bus follows the addressing standards outlined in the TEKNOFEST technical guide [15]. This setup ensures synchronized operation across system components and enhances overall data flow. Peripheral design was also supported via a GitHub repository [16].

Wishbone's advantages — such as low cost, simplicity, and efficiency — make it especially suitable for FPGA-based and resource-constrained systems. As an open-source protocol, it is easy to adapt to specific design needs. Compared to more complex alternatives like TileLink or AMBA, Wishbone uses fewer control signals and hardware resources. Its modularity allows for straightforward integration with various peripherals, enabling low-latency, high-speed communication between the processor and external modules, thereby boosting overall system performance.

This paragraph discusses the features and operational principles of the UART module added to the processor. UART (Universal Asynchronous Receiver–Transmitter) has been integrated as an interface that allows communication between the processor and the external world, enabling data exchange via the RX (Receiver) and TX (Transmitter) pins. The communication between the processor core and this module is managed using the WISHBONE bus. The WISHBONE bus is a standard data bus that enables peripheral devices to work in a compatible and reliable manner with the processor.

The memory map for the UART module is detailed in Table 3 This memory map includes four fundamental registers used for the control and functionality of the UART:

(1) uart_ctrl (Control Register): This register is used to configure the UART module and set operational conditions, such as initializing the device, setting baud rates, and assigning other control parameters. It is accessible for write operations.
(2) uart_status (Status Register): This register shows the current status of the UART, indicating whether data has been received or if the transmission has been completed successfully. It is read-only.
(3) uart_rdata (Receive Data Register): This register stores the data received via the RX line. It allows the processor to read the data from the UART module and is read-only.
(4) uart_wdata (Write Data Register): This register is used to send data via the TX line. It allows write operations for the processor to transmit data to the external world. These structures enable the flexible integration of the UART module into the processor and ensure reliable execution of various data communication tasks. As a result, the processor can effectively communicate with different peripheral devices and take advantage of the fast, asynchronous data transfer provided by UART.

Table 4
Memory map of SPI.

Address	Name	Explanation
0x20010000	spi_ctrl	Control register
0x20010004	spi_status	Status register
0x20010008	spi_rdata	Data read register
0x2001000c	spi_wdata	Data write register
0x20010010	spi_cmd	Command register

Table 5
Memory map of PWM.

Address	Name	Explanation
0x20020000	pwm_control	PWM control register
0x20020004	pwm_period	PWM period register
0x20020008	pwm_period	PWM threshold register
0x2002000c	pwm_step	PWM step register

The baud rate is calculated according to the formula provided above. When programming UART using assembly language, it is necessary to calculate the baud rate value based on this formula to ensure the correct data transmission speed. This calculation ensures that the UART module operates at the correct speed, enabling compatible data communication between the processor and peripheral devices.

$$f_{\text{baud}} = \frac{f_{\text{clk}}}{\text{baud_div} + 1} \tag{1}$$

To enable the processor core to support different communication protocols, an SPI (Serial Peripheral Interface) module has been integrated into the system. The SPI module has five different register structures to manage data communication, and these registers are organized according to a memory map shown in Table 4. In this memory map, access to the relevant registers is provided through specific addresses.

These registers play a critical role during operations performed using the SPI protocol and regulate data exchange between the processor and peripheral devices. The SPI module integrated into the designed processor is equipped with various operating modes and features. These modes and features will be detailed and explained in sub-sections.

To obtain a PWM (Pulse Width Modulation) signal from the processor, a PWM module has been integrated into the system. This module is connected to the WISHBONE bus protocol to operate in harmony with the processor core. Access to various control and data registers of the PWM module is provided through the addresses shown in Table 5.

These addresses allow the processor to access and configure the module's settings and operating parameters. The PWM module includes critical registers to control essential characteristics of the PWM signal, such as signal width, frequency, and duty cycle. This structure enables the processor to send PWM signals to peripheral devices or applications that require control, making the system suitable for a wide range of applications.

4. Experimental results

4.1. Synthesis and implementation

All experiments were conducted using the Zybo Z7-20 FPGA board. The processor frequency was set to 40 MHz using the clock wizard. Timing issues arise at higher frequencies. In general tests, a Wrapper module surrounding the processor was used [17]. This module allows external commands to be loaded into memory via UART when the processor's bitstream is loaded onto the board. The RAM latency in the Wrapper was set to 2. Additionally, a performance counter is present within the Wrapper, and benchmark tests were calculated using this counter.

Fig. 12, Fig. 13 and Table 6 present timing analysis and resource utilization data related to the Vivado implementation. In Fig. 12, the



Fig. 12. Implementation result of processor according to Zybo Z7-20.

Table 6
Resource usage.

Resource	Utilization	Available	Utilization
LUT	7534	53 200	14.16
LUTRAM	984	17 400	5.66
FF	3967	106 400	3.73
BRAM	133	140	95.00
IO	12	125	9.60
BUFG	3	32	3.38
MMCM	1	4	25.00

details of the timing analysis obtained after the implementation on the FPGA are shown. This analysis includes all critical paths and delays necessary for the design to operate within the required timing. Fig. 13 only presents the resource utilization visualization as the output from Vivado. This visual shows in detail how much resource the design uses on the FPGA, including used LUTs, flip-flops, and other hardware resources. Finally, Table 6 illustrates how much the performance of the design has been optimized with a more detailed timing analysis. Such analyses are crucial for improving the efficiency of FPGA-based designs and preventing timing errors.

4.2. Energy consumption

Table 7 shows the energy consumption results. The experimental evaluation of our RISC-V processor reveals a total on-chip power consumption of 0.268 W, comprising 0.153 W dynamic power and 0.116 W static power. Thermal analysis indicates a maximum ambient temperature of 81.9 °C and a junction temperature of 28.1 °C, suggesting efficient thermal management. However, the confidence level of these measurements is reported as low, indicating the need for further validation.

Comparative studies with other RISC-V processors highlight the efficiency of our design. For instance, the RisCO2 processor, optimized for energy consumption, operates at 0.29 mJ per instruction, which is approximately 53.5% and 63.2% less energy-intensive than the CV32E40P and Ri5cy processors, respectively [18]. Similarly, a study on an energy-efficient single-cycle RV32I microprocessor reports a power consumption of 14.7 mW at 50 MHz, demonstrating the potential for low-power designs in edge computing applications [19].

Our processor's power consumption aligns with these efficient designs, positioning it as a competitive option for resource-constrained embedded systems. The low static and dynamic power consumption, coupled with effective thermal management, underscores the suitability of our processor for applications where energy efficiency is paramount. However, the low confidence level in the reported measurements necessitates additional testing to confirm these findings and ensure the processor's reliability under various operational conditions.

4.3. RISCOF architecture test

To install the RISCOF testing tool in the Ubuntu environment, a suitable directory structure was first created, and the necessary files were transferred to the local system. This process was carried out using specific commands to ensure the system was properly configured. The kernel files were placed in a designated directory, avoiding cache

Table 7
Power consumption.

Parameters	Values
Total On-Chip Power (W)	0.268
Design Power Budget (W)	Unspecified
Power Budget Margin (W)	NA
Dynamic Power (W)	0.153
Device Static Power (W)	0.116
Effective TJA (C/W)	11.5
Max Ambient Temperature (°C)	81.9
Junction Temperature (°C)	28.1
Confidence Level	Low

usage. The wrapper developed for the Teknofest competition was appropriately updated to export data from the processor core. This export process ensures that the data is saved in the .signature format. To run the wrapper, the Iverilog simulator was used. A Makefile configuration was prepared to compile the RISCOF commands, ensuring the test process works smoothly. Fig. 14 shows a successful RISCOF result.

4.4. Uart test

To evaluate the UART communication capability of the processor we developed, a basic UART code was written using the RISC-V GNU Compiler Collection (GCC) compiler. This code is designed to test whether the processor can perform data transmission and reception via the UART protocol. The code written in this study will be compiled and executed on the processor, and the accuracy of data transmission via UART will be evaluated.

The details of the code and the related data flows are presented in Fig. 15. This figure summarizes the working principles of the application and the connection between the processor and the UART unit. The purpose of the test is to verify that the processor can successfully establish data communication with external peripheral devices via UART.

To validate the UART communication capability of the processor we developed, the basic UART code written with the RISC-V GCC compiler was successfully executed on the processor. This code was compiled to test the processor's ability to send and receive data using the UART protocol.

The test results showed that the processor could correctly transmit and receive data over UART, which was confirmed by the terminal output in Fig. 16. The terminal screen displays successful data transmission results, indicating that the UART communication occurred without errors. This test is considered a crucial step in confirming the processor's ability to establish reliable communication with external peripheral devices.

4.5. SPI test

To evaluate the SPI communication capability, it was planned to establish an SPI communication between the Zybo board and the STM32 Bluepill board. In this study, the Zybo board is configured as the "master" and the STM32 Bluepill board as the "slave".

The necessary test code for establishing SPI communication between the two boards has been written and is presented in Fig. 17. The test aims to evaluate the accuracy of data transmission and synchronization between the two boards. This test serves as a fundamental reference for assessing the reliability of SPI-based applications used in embedded systems.

In the SPI communication test, it was observed that the communication between the Zybo board and the STM32 Bluepill board was successful. During the test, the data sent by the Zybo board was transmitted to the STM32 Bluepill board, and a LED on the STM32 board was lit. This test demonstrates that data transmission via the

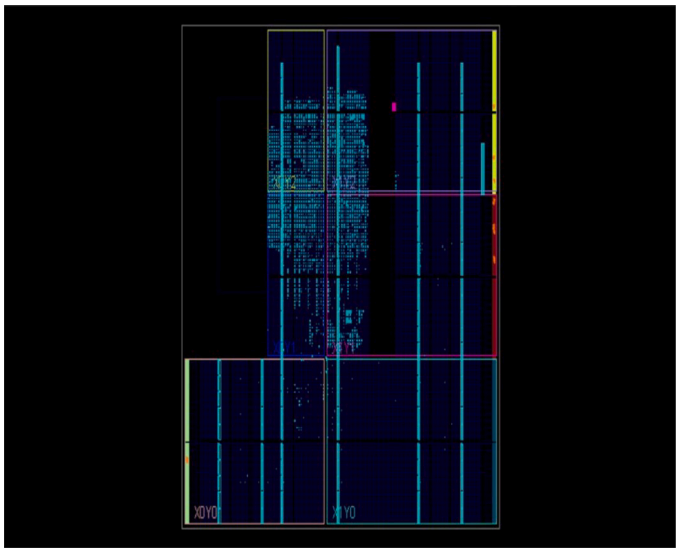


Fig. 13. Placed resources.

```
kamer@DESKTOP-7ULN6M1:
INFO | TEST NAME                                                    : COMMIT ID                                                    : STATUS
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/add-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/addi-01.S : -                                               : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/and-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/andi-01.S : -                                               : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/auipc-01.S : -                                              : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/beq-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/bge-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/bgeu-01.S : -                                               : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/blt-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/bltu-01.S : -                                               : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/bne-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/fence-01.S : -                                               : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/jal-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/jalr-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/lb-align-01.S : -                                            : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/lbu-align-01.S : -                                           : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/lh-align-01.S : -                                            : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/lhu-align-01.S : -                                           : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/lui-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/lw-align-01.S : -                                            : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/misalign1-jalr-01.S : -                                      : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/or-01.S : -                                                  : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/ori-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/sb-align-01.S : -                                            : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/sh-align-01.S : -                                            : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/sll-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/slli-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/slt-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/slti-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/sltiu-01.S : -                                               : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/sltu-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/sra-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/srai-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/srl-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/srli-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/sub-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/sw-align-01.S : -                                            : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/xor-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/I/src/xori-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/M/src/div-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/M/src/divu-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/M/src/mul-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/M/src/mulh-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/M/src/mulhsu-01.S : -                                               : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/M/src/mulhu-01.S : -                                                : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/M/src/rem-01.S : -                                                 : Passed
INFO | /home/kamer/KamerSoCC/riscv-arch-test/riscv-test-suite/rv32i_m/M/src/remu-01.S : -                                                : Passed
INFO | Test report generated at /home/kamer/KamerSoCC/riscof_work/report.html.
INFO | Opening test report in web-browser
```

Fig. 14. Commands that pass the test in the RISCOF environment.

```
test1 > C main1.c
1
2   #include "uart.h"
3
4
5
6   int main(){
7
8     init_uart();
9     ee_printf("Merhaba Bu Bir UART Testidir.");
10
11  }
12
13
14
```

Fig. 15. Test code of UART.

SPI protocol occurred correctly and that the STM32 board functioned properly as an external peripheral.

A photo proving the success of the test is shown in Fig. 18. This visual depicts the moment when the LED on the STM32 Bluepill board is lit by the data sent from the Zybo board. This result confirms that the processors and peripheral units are working in harmony with the SPI communication protocol.

4.6. PWM test

The tests were conducted using two different software codes. During these tests, the goal was to generate signals with two different duty cycle values using the PWM module integrated into the processor. In Fig. 19(a), it is noted that the signal observed on the oscilloscope has a 50% duty cycle. On the other hand, in Fig. 19(b), the result of a different test successfully produced a PWM signal with a 20% duty cycle. These results demonstrate that the PWM module can accurately generate signals with the desired duty cycle values.

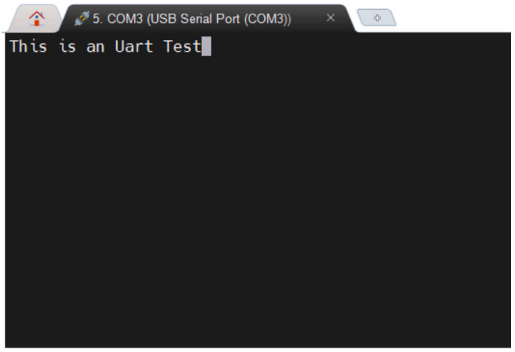


Fig. 16. Result of UART test.

```c
#include "printf.h"
#include "spi_driver.h"

int main(){

init_uart();
    SPI_WDATA = 0x4B;
    SPI_CTRL = 0x000F0001;
    SPI_CMD = 0x00002000;
    while(!((SPI_STATUS>>1)%2));

    while(1);
}
```

Fig. 17. Test code of SPI.

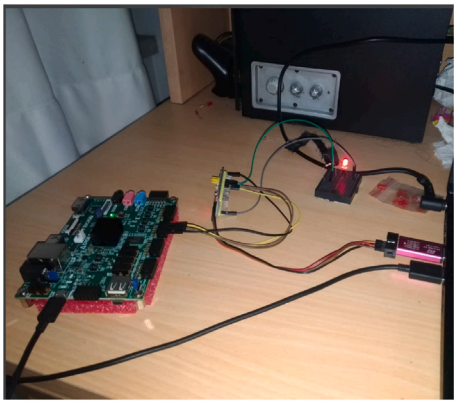


Fig. 18. SPI test with Zybo Z7-20.

4.7. Dhrystone benchmark

To evaluate the performance of our processor, the Dhrystone Benchmark test was applied. One of the key parameters used in this test, the Number_of_runs value, was set to 3,000,000, and the processor was run at a frequency of 40 MHz to initiate the test. Once the Dhrystone test was completed, the difference in time between the stop_time_val and start_time_val values was calculated and recorded as 3,381,001,881. The USER_TIME value displayed in the terminal output represents the number of cycles completed by the processor during the test. Based on the data obtained during the test process, the Dhrystone Benchmark results were evaluated according to the following formulas:

Dhrystone Per Second: This refers to the number of Dhrystone operations performed by the processor per second and is calculated as follows:

$$\text{Dhrystone Per Second} = \frac{\text{Hz} \times \text{Number of Runs}}{\text{User Time}} \quad (2)$$

DMIPS: The DMIPS (Dhrystone MIPS) value, used for comparing system performance, is obtained by dividing the Dhrystone Per Second value by 1,757:

$$\text{DMIPS} = \frac{\text{Dhrystone Per Second}}{1757} \quad (3)$$

DMIPS/MHz: It is the performance measurement of the processor normalized to frequency:

$$\text{DMIPS/MHz} = \frac{\text{DMIPS}}{\text{System Frequency}} \quad (4)$$

As a result of the above calculations, a value of 0.5050 DMIPS/MHz has been obtained for our processor. The test results, shown in hexadecimal format in Fig. 20, provide an important indication for evaluating the performance of our processor [20,21].

4.8. Coremark benchmark

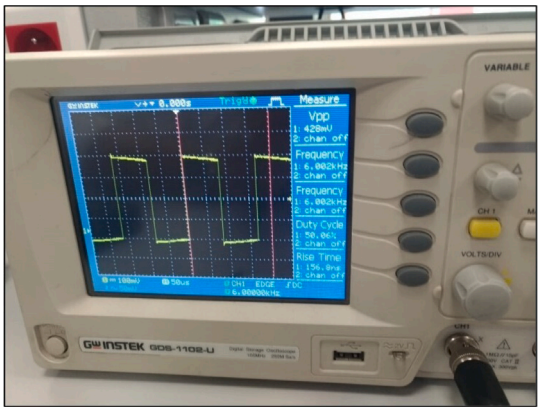
A CoreMark test with 5,000 iterations was conducted to evaluate the performance of our processor. This test reliably measures the processor's working capacity by using different algorithms and data manipulations to assess the efficiency of the processor under specific workloads. As a result of the test, a value of 1.35 CoreMark/MHz was obtained for our processor. This result demonstrates the processor's processing capacity per unit frequency, proving its performance in embedded systems. The value of 1.35 CoreMark/MHz indicates that our processor offers effective performance in real-time applications and can be confidently used in computation-intensive tasks. Fig. 21 shows the Coremark result.

In this study, CoreMark and Dhrystone benchmark tests were used to evaluate the performance of our processor. CoreMark is a widely used benchmark test that measures the efficiency of a processor in mathematical and logical computations. Dhrystone, on the other hand, evaluates the processing power in integer calculations. The results of these tests have helped us determine the overall efficiency of our processor.

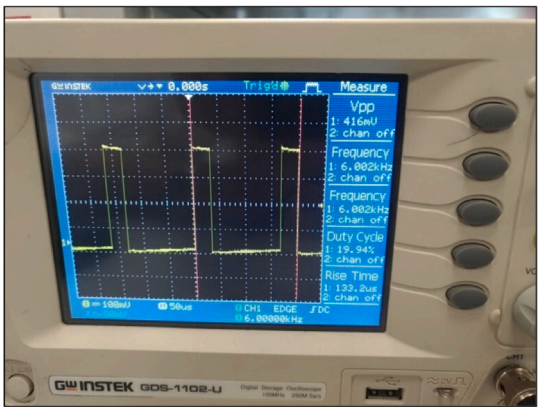
As a result of the CoreMark test, our processor achieved a score of 1.35 CoreMark/MHz, while the Dhrystone test was measured at 0.505 DMIPS/MHz. These values indicate that our processor provides efficient performance in terms of basic functionality and performs at an average level when compared to other cores evaluated by industry standards. Moreover, these tests validate the processor's core computational power and efficiency, particularly its capabilities in logical and integer calculations.

5. Results

This processor, implemented on a Zybo Z7-20 FPGA board, achieves a performance of 1.35 CoreMark/MHz and 0.505 DMIPS/ MHz while utilizing only 7534 LUTs and 3967 FFs. Compared to widely known open-source cores like CV32E40P, Ri5cy, or VexRiscv, our processor offers a balanced trade-off between performance, area, and resource usage. While CV32E40P consumes around 49 mW with 9072 LUTs, and Ri5cy uses 11,912 LUTs, our design achieves comparable results with significantly lower resource demand. Moreover, UART, SPI, and PWM functionalities have been validated via physical hardware interaction, not only simulation. These contributions differentiate our processor from existing works. In this study, a 32-bit RISC-V processor architecture optimized for area efficiency was designed and implemented on the Zybo Z7-20 FPGA board. The resource utilization of the processor is summarized in Table 6. The processor utilizes 14.16% LUT, 5.66% LUTRAM, and 3.73% FF, demonstrating a compact area footprint. Additionally, the processor uses 95% BRAM and 9.6% IO, reflecting efficient use of FPGA resources.



(a) 50% Duty Cycle PWM signal.



(b) 20% Duty Cycle PWM signal.

Fig. 19. (a) 50% Duty Cycle PWM (b) 20% Duty Cycle PWM.

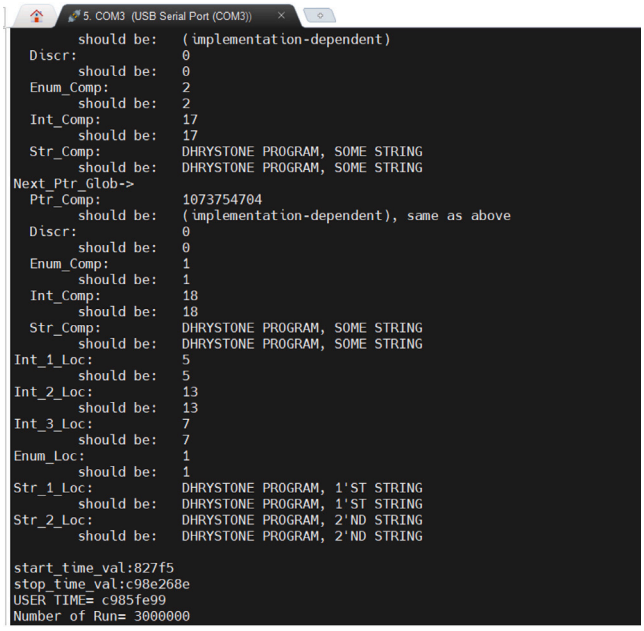


Fig. 20. Dhrystone benchmark result.

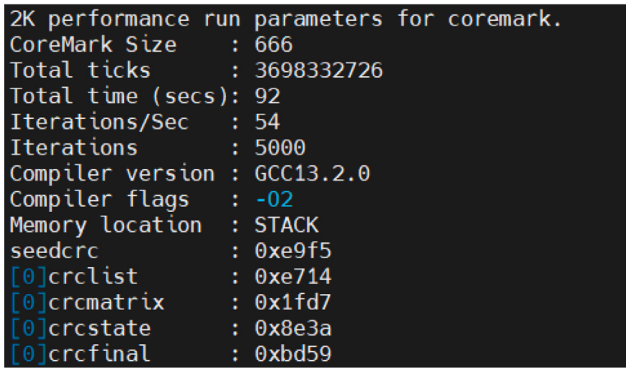


Fig. 21. CoreMark result.

Compared to other RISC-V processors reported in the literature, the area efficiency of our design is notable. For instance, the RisCO2 processor, which is optimized for energy consumption, operates with 0.29 mJ per instruction, while using 4889 LUT and 2354 FF. This results in approximately 53.5% and 63.2% less energy consumption than the CV32E40P and Ri5cy processors, respectively [18]. In contrast, these processors exhibit higher LUT and FF usage, leading to larger area footprints.

Table 8
RISC-V processors synthesis and implementation results with FPGA.

Core	ISA	Pipeline	LUT	FF	Power
Micro-riscy	RV32E	2	2225	1276	15
Zero-riscy	RV32IM	2	3171	1928	20
Ri5cy	RV32IMF	4	11,912	4249	52
CV32E40P	RV32IMF_Zfinx	4	9072	2553	49
RisCO2	RV32E_Zfinx	5	4889	2354	14
This Work	RV32IM	4	7534	3967	268

The energy consumption of our processor is measured at 0.268 W, which is slightly higher than the RisCO2 processor. However, the performance is significantly better. The processor achieved 0.505 DMIPS/MHz in the Dhrystone benchmark test and 1.35 CoreMark/MHz in the CoreMark test. This demonstrates that our processor provides high performance despite its low power consumption and compact design.

During the testing, other processors were tested with the Nexy-A4 FPGA. The FPGA board we used, the Zybo Z7-20, has the same nanometer architecture. It also features an ARM processor. However, during synthesis, the ARM processor was not included and the board was used solely as an FPGA. The comparison is made for this reason.

Furthermore, peripheral tests have been successfully conducted. The UART communication was tested and completed without issues. An SPI test was performed between the processor and an STM32 microcontroller board, successfully transmitting a hexadecimal value (0x4B) to light an LED. In the PWM test, the processor generated a signal at a frequency of 6 kHz, with the threshold values adjusted using an oscilloscope.

In conclusion, the area efficiency, low power consumption, high performance, and successful peripheral functionality of our processor make it a suitable choice for resource-constrained embedded systems. However, the low confidence level in the reported measurements, as presented in Table 8, indicates the need for additional testing to ensure long-term reliability and robustness.

6. Discussion

In this study, the RISC-V-based processor operating at 40 MHz achieved scores of 1.35 CoreMark and 0.505 DMIPS/MHz, demonstrating an efficient design that utilized 7534 LUTs, 3967 BRAMs, 133

Table 9
Resource usage comparison of our risc-v processor and other evaluated cores.

Logic Elements	Flute	Orca	Piccolo	PicoRV32	SweRV	Taiga	VexRiscv	This Work
LUT	13 537	3917	13 324	4143	30 340	4504	3777	7534
REGISTERS	11 343	3100	10 454	3363	16 261	3176	3305	3967
BRAM	34.5	32	33.5	32	62	34	33	133

BRAM cells, and 984 LUTRAMs. The successful operation of the UART, SPI, and PWM units highlights the processor's capability for peripheral integration, while passing all instruction tests using the RISCOF framework confirms its RISC-V compliance. Future plans include implementing a physical design via OpenLane, which will enhance the processor's applicability for ASIC implementations.

According to the data presented in Table 9, our designed processor demonstrates a balanced resource utilization profile. With 7534 LUTs and 3967 registers, it occupies a moderate area, while consuming significantly fewer resources compared to some cores-for instance, SweRV uses 30,340 LUTs. The BRAM usage appears relatively high at 133; however, this is primarily due to memory blocks included in the wrapper surrounding the processor core rather than the core itself. Therefore, the actual processor design is more compact in terms of memory utilization. Overall, our processor strikes a suitable balance between area efficiency and expandable memory structure, making it a strong candidate for embedded system applications.

The compared processors are RISC-V compatible cores widely referenced in the open-source community and literature, where performance metrics are publicly available. These cores are used as benchmarks in academic and commercial environments for their resource usage, performance, and scalability. Since our processor targets similar hardware constraints, comparisons with these processors are meaningful and relevant.

Additionally, some of the compared processors were tested on the PYNQ FPGA development board, which, like our Zybo Z7-20 board, contains the Xilinx XC7Z020 FPGA chip. This hardware similarity ensures that the performance metrics obtained can be compared more fairly and directly. Thus, the evaluation minimizes inconsistencies due to hardware variations and better reflects the actual performance of our processor.

6.1. Contributions of this work

(1) A compact 32-bit RISC-V processor core based on RV32IM with a four-stage pipeline, designed for low-area and low-power FPGA implementations.
(2) Integration and real hardware testing of UART, SPI, and PWM peripherals using the Wishbone bus, which is rare in academic designs.
(3) Use of Unsigned Binary Multiplication and Non-Restoring Division algorithms with FSM control for minimal logic utilization.
(4) Functional validation using real hardware benchmarks including CoreMark (1.35 CoreMark/MHz), Dhrystone (0.505 DMIPS/MHz), and RISCOF.
(5) Compatibility with OpenRAM and OpenLane platforms for future ASIC realization.
(6) Operations that are typically implemented using a five-stage pipeline architecture have been realized in this study with a four-stage pipeline. This design choice was made to achieve a more compact and area-efficient processor architecture.
(7) A detailed performance and area comparison has been conducted against several open-source RISC-V cores. Unlike previous works, our design combines cache structures, real peripheral support, and benchmark results into a single compact processor core validated through hardware.

Future directions include enhancing the processor's capabilities by adding support for the Control and Status Register (CSR) module and branch prediction mechanisms, as well as extending the design to support OpenLane GDSII generation for full-chip tape-out via open-source ASIC flow tools.

CRediT authorship contribution statement

Kamer Kırali: Validation, Software, Methodology, Investigation, Conceptualization. **Can Bülent Fidan:** Supervision, Project administration, Funding acquisition.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used ChatGPT in order to improve the language and readability of the manuscript. After using this tool, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors would like to express their sincere gratitude to the Karabük University Scientific Research Projects Coordination Unit for their financial support of the project, KBUBAP-23-YL-034.

References

[1] A. Waterman, K.A. Asanovic, The RISC-V instruction set manual volume I: User-level ISA document version 2.2 editors, 2017.
[2] L. Poli, S. Saha, X. Zhai, K.D. McDonald-Maier, Design and implementation of a RISC V processor on FPGA, in: Proceedings - 2021 17th International Conference on Mobility, Sensing and Networking, MSN 2021, Institute of Electrical and Electronics Engineers Inc., 2021, pp. 161–166, http://dx.doi.org/10.1109/MSN53354.2021.00037.
[3] The-OpenROAD-Project/OpenLane: OpenLane is an automated RTL to GDSII flow based on several components including OpenROAD, Yosys, Magic, Netgen and custom methodology scripts for design exploration and optimization. URL: https://github.com/The-OpenROAD-Project/OpenLane.
[4] OpenRAM | An open-source static random access memory (SRAM) compiler. URL: https://openram.org/.
[5] Y. Chang, Y. Liu, C. Peng, J. Guo, Y. Zhao, Design of a configurable five-stage pipeline processor core based on RV32IM, Electronics 13 (2023) 120, http://dx.doi.org/10.3390/ELECTRONICS13010120, https://www.mdpi.com/2079-9292/13/1/120/htm, 2024, https://www.mdpi.com/2079-9292/13/1/120.
[6] F.W. Wibowo, Comparison of multiplication algorithms based on FPGA, in: 2018 2nd Borneo International Conference on Applied Mathematics and Engineering, BICAME 2018, Institute of Electrical and Electronics Engineers Inc., 2018, pp. 326–331, http://dx.doi.org/10.1109/BICAME45512.2018.1570505372.
[7] P. Kanjarbhat, P. Chandaragi, U.V. Wali, Implementation of PWM using RISC-V processor, in: 2024 International Conference on Advances in Modern Age Technologies for Health and Engineering Science, AMATHE 2024, Institute of Electrical and Electronics Engineers Inc., 2024, http://dx.doi.org/10.1109/AMATHE61652.2024.10582142.
[8] M. Awedh, A. Mueen, Research article design and FPGA implementation of UART using microprogrammed controller.
[9] N. Safari, A. Joudi, M. Rajabalipanah, Z. Jahanpeima, H. Sadeghzadeh, Z. Navabi, HIRMA: High-performance implementation for RISC-V microcontroller applications, in: 2023 IEEE East-West Design and Test Symposium, EWDTS 2023 -Proceedings, Institute of Electrical and Electronics Engineers Inc., 2023, http://dx.doi.org/10.1109/EWDTS59469.2023.10297033.

[10] C. Heinz, Y. Lavan, J. Hofmann, A. Koch, A catalog and in-hardware evaluation of open-source drop-in compatible RISC-V softcore processors, in: 2019 International Conference on Reconfigurable Computing and FPGAs, ReConFig 2019, Institute of Electrical and Electronics Engineers Inc., 2019, http://dx.doi.org/10.1109/RECONFIG48160.2019.8994796.

[11] M. Gazziro, J.M. de Assumpção Junior, O.H.A. Junior, M.R. Cavallari, J.P. Carmo, Design and evaluation of open-source soft-core processors, Electronics 13 (2024) 781, http://dx.doi.org/10.3390/ELECTRONICS13040781, https://www.mdpi.com/2079-9292/13/4/781/htm, https://www.mdpi.com/2079-9292/13/4/781.

[12] 1. Introduction — RISCOF 1.24.0 documentation. URL: https://riscof.readthedocs.io/en/stable/intro.html.

[13] S. Kaur, M. Manna, R. Agarwal, VHDL implementation of non restoring division algorithm using high speed adder/subtractor, Int. J. Adv. Res. Electr. Electron. Instrum. Eng. 2 (2013) 3317–3324.

[14] X. Zhang, W. Wolf, A survey of cache architectures in embedded processors, ACM Trans. Embed. Comput. Syst. (TECS) 3 (2) (2004) 234–263.

[15] ÇİP tasarim yariŞmasi Şartnamesİ - pdf Ücretsiz indirin. URL: https://docplayer.biz.tr/226586067-Cip-tasarim-yarismasi-sartnamesi.html.

[16] Kasirga-kizil/tekno-kizil: Kasirga - kizil takımı teknofest 2023 Çip tasarımı - kizil İşlemci Projesi. URL: https://github.com/KASIRGA-KIZIL/tekno-kizil.

[17] Tutel-tubitak/teknofest_2023_cip_tasarim_yarismasi: Bu depo teknofest 2023 Çip tasarım yarı şması'nda analog tasarım ve sayı sal İşlemci tasarı mı kategorilerinde çeşitli dosyaları paylaşmak için kullanılacaktır. URL: https://github.com/TUTEL-TUBITAK/TEKNOFEST_2023_Cip_Tasarim_Yarismasi.

[18] R. Núñez Prieto, D. Castells-Rufas, L. Terés-Terés, RisCO2: Implementation and performance evaluation of RISC-V processors for low-power CO2 concentration sensing, Micromachines 14 (7) (2023) http://dx.doi.org/10.3390/mi14071371, URL: https://www.mdpi.com/2072-666X/14/7/1371.

[19] S. Shukla, P.K. Jha, K.C. Ray, An energy-efficient single-cycle RV32I microprocessor for edge computing applications, Integration 88 (2023) 233–240, http://dx.doi.org/10.1016/j.vlsi.2022.09.005, URL: https://www.sciencedirect.com/science/article/pii/S0167926022001195.

[20] Versal dhrystone benchmark user guide — Embedded design tutorials 2021.1 documentation. URL: https://xilinx.github.io/Embedded-Design-Tutorials/docs/2021.1/build/html/docs/User_Guides/Performance_Benchmark/Dhrystone/README.html.

[21] Performance calculation • versal dhrystone benchmark user guide (XD071) • reader • AMD technical information portal. URL: https://docs.amd.com/r/en-US/xd071-versal-dhrystone-benchmark/Performance-Calculation.